

Extra Space Asia — Recommendation & Booking API

Build chat / web / mobile experiences that quote and reserve self-storage units through Extra Space Asia.

- **Version** — 1.2.0 (2026-05-12) — see [§13 · Changelog](#)
- **Base URL** — `https://backend.extraspace.com.sg`
- **Auth** — X-API-Key: `esa_<prefix>.<secret>` on every request
- **Format** — JSON in/out, UTF-8, ISO dates (YYYY-MM-DD), decimals as numbers (91.01)
- **Status** — current scope is **recommend** → **reserve** → **confirm price**. Lease creation (move-in) is handled by Extra Space Asia operations and is not part of this integration.

Heads up: the recommender is **stateful**. Read [§3 — Identity](#) before writing your first integration. Four context fields glue every step together — get them right and everything just works.

Table of contents

1. [Quick-start \(copy-paste\)](#)
2. [Booking lifecycle](#)
3. [Identity model](#)
4. [POST /api/recommendations](#)
5. [Multi-turn search \(3 levels deep\)](#)
6. [POST /api/reservations/reserve](#)
7. [GET /api/reservations/move-in/cost](#)
8. [Reservation lifecycle \(read/update/cancel\)](#)
9. [End-to-end worked example](#)
10. [Reference tables](#)
11. [Best practices](#)
12. [Support](#)
13. [Changelog](#)

1 · Quick-start

Five-minute integration test:

```
# 1. Get a quote (3 priced unit slots)
curl -X POST https://backend.extraspace.com.sg/api/recommendations \
  -H "X-API-Key: $ESA_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "duration_months": 6,
    "filters": { "location": ["L017"], "size_range": ["30-35"] },
    "context": {
      "channel": "chatbot",
      "request_id": "000000000-0000-0000-0000-0000000000001",
      "session_id": "demo-session-1",
      "customer_id": "demo-customer-1"
    }
  }'
```

You'll get back **3 slots** (Best Match / Best Alternative / Best Price) with full pricing, per-month breakdowns, and continuation tokens. From there:

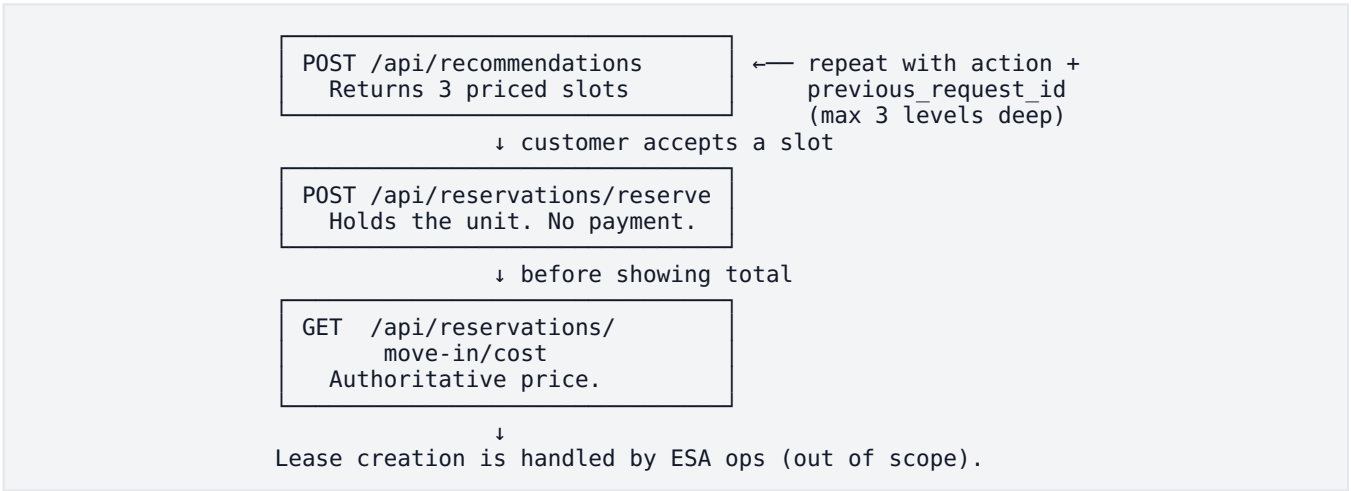
- Customer accepts a slot → `POST /api/reservations/reserve` (\$6)
- Show authoritative price → `GET /api/reservations/move-in/cost` (\$7)
- Customer wants different options → recommend again with `previous_request_id` (\$5)

Rate limits

Endpoint	Limit
<code>POST /api/recommendations</code>	120/min
<code>POST /api/reservations/reserve</code>	10/min
<code>GET /api/reservations/move-in/cost</code>	30/min
<code>PUT /api/reservations/<id> (update/cancel)</code>	10/min

HTTP 429 when over, with `retry_after` seconds in the body.

2 · Booking lifecycle



Optional reservation management between **reserve** and lease completion:

Method	Path	Purpose
GET	<code>/api/reservations/<waiting_id></code>	Read state
PUT	<code>/api/reservations/<waiting_id></code>	Modify (move-in date, contact info)
PUT	<code>/api/reservations/<waiting_id>/cancel</code>	Cancel

3 · Identity model

Four context fields are required on every recommend call. They tie every step of one customer's flow together — from first quote through follow-up turns to the held reservation.

Field	Format	Lifetime	Source
<code>channel</code>	enum: chatbot web api admin	per call	hardcoded by your integration
<code>request_id</code>	string ≤ 64 chars (UUID v4 recommended)	unique per turn	bot mints fresh per call
<code>session_id</code>	string ≤ 64 chars	per conversation — same on every recommend turn AND on / reserve	bot mints once at chat start
<code>customer_id</code>	string ≤ 64 chars	per customer lifetime	your channel's stable user id (or stable surrogate for anonymous flows)

Plus one optional field for follow-up turns:

Field	When to send
previous_request_id	The request_id of the prior recommend turn. Tells the engine "this is a continuation". See §5.

Validation rules - All four core fields are mandatory; missing/empty/whitespace-only → HTTP 400. - Length cap: 64 chars per field. - Case-sensitive. - Server returns next_turn.request_id + next_turn.session_id in the response so you can confirm what got logged.

4 • POST /api/recommendations

Quote engine. Returns **up to 3 priced slots** matching the customer's intent.

Minimal request

```
curl -X POST https://backend.extraspace.com.sg/api/recommendations \
-H "X-API-Key: $ESA_API_KEY" \
-H "Content-Type: application/json" \
-d '{
  "duration_months": 6,
  "filters": { "location": ["L017"] },
  "context": {
    "channel": "chatbot", "request_id": "<uuid>",
    "session_id": "<uuid>", "customer_id": "user-42"
  }
}'
```

Full request body

```
{
  "mode": "recommendation",
  "duration_months": 6,
  "filters": {
    "location": ["L017"],
    "unit_type": ["W", "L"],
    "climate_type": ["A", "AD"],
    "size_range": ["30-35"],
    "unit_id": [107197],
    "concession_id": 11872,
    "plan_id": [10],
    "coupon_code": "PROMO123"
  },
  "context": {
    "channel": "chatbot",
    "request_id": "550e8400-e29b-41d4-a716-446655440000",
    "session_id": "sess-conv-7",
    "customer_id": "user_xyz_123",
    "previous_request_id": "...",
    "picked_slot": 1,
    "action": "more_like_this"
  },
  "constraints": {
    "max_distance_km": 50,
    "exclude_unit_ids": [107197]
  }
}
```

Field	Required	Notes
mode	optional	recommendation (default) or quote (single unit; requires filters.unit_id)
duration_months	yes	integer 1-12 (SiteLink's technical limit; longer leases are renewals)
filters.location	yes	array of site codes
filters.unit_type	optional	W/WN/L/U/M/LL (see §10)
filters.climate_type	optional	A/AD/NC/RF (see §10)
filters.size_range	optional	sqft buckets like "30-35"
filters.unit_id	optional	array of integers — pin to specific unit(s)
filters.unit_name	optional	array of strings — customer-friendly unit number/name (e.g. ["4120"], ["Locker A1"]). Resolved to unit_id via the (filters.location, sUnitName) join. Useful when the customer says "unit 4120" and the bot doesn't know the internal id. Names may collide across sites — always paired with filters.location.
filters.plan_id	optional	array of integers — cross-site brand filter (one plan covers multiple per-site concessions)
filters.concession_id	optional	single integer — pins a specific (unit, plan, concession) tuple
filters.coupon_code	optional	string — gates coupon-only plans
context.*	mostly yes	see §3
constraints.max_distance_km	optional	overrides default neighbour radius (50 km)
constraints.exclude_unit_ids	optional	up to 200 ids — won't be shown

Sample response

```

{
  "mode": "recommendation",
  "request_id": "550e8400-e29b-41d4-a716-446655440000",
  "served_at": "2026-05-04T01:23:45Z",

  "stats": {
    "recommendation_level": 1,
    "candidates_pool_size": 14,
    "relax_strategy_used": "none",
    "saturation_signal": false
  },

  "slots": [
    {
      "slot": 1, "label": "Best Match",
      "unit_id": 107197, "facility": "L017",
      "unit_type": "W", "climate_type": "AD",
      "size_range": "30-35", "size_sqft": 32.5,
      "plan_id": 10, "plan_name": "Moving Season SG",
      "concession_id": 11872, "concession_name": "SS-TAC-Move-R-30%",
      "discount_summary": "30% off every month",

      "match_flags": {
        "alternative_strategy": null,
        "relaxed_dims": [],
        "distance_km": 0,
        "travel_warning": false
      },

      "terms": {
        "discount_perpetual": true,
        "prepayment_months": 6,
        "post_prepay_uplift_pct": 5.0,
        "lock_in_months": 0,
        "promo_valid_until": "2026-12-31"
      },

      "reservation_fee": 50000.0,
      "reservation_fee_source": "override",

      "pricing": {
        "first_month_total": 238.71,
        "total_due_at_movein": 693.76,
        "total_contract": 693.76,
        "monthly_average": 115.63,
        "rate_during_prepay": 80.50,
        "rate_after_prepay": 84.52,
        "rate_change_date": "2026-11-15",
        "monthly_all_in_during_prepay": 91.01,
        "monthly_all_in_after_prepay": 95.40,
        "monthly_insurance_premium": 3.00,
        "monthly_insurance_tax": 0.27,
        "breakdown": [
          { "month_index": 1, "billing_date": "2026-05-15",
            "rent": 115.00, "discount": 34.50, "rent_proration_factor": 1.0,
            "insurance": 3.00, "deposit": 296.00, "admin_fee": 30.00,
            "rent_tax": 7.25, "insurance_tax": 0.27, "total": 238.71 },
          { "month_index": 2, "billing_date": "2026-06-15",
            "rent": 115.00, "discount": 34.50, "insurance": 3.00,
            "deposit": 0, "admin_fee": 0, "total": 91.01 }
          /* ... one entry per month of duration_months ... */
        ]
      },

      "insurance": {
        "selected": { "id": 1, "coverage": 1000, "premium": 3.00 },
        "options": [
          { "id": 1, "coverage": 1000, "premium": 3.00 },
          { "id": 2, "coverage": 2000, "premium": 7.00 }
        ],
        "min_required": 1000
      },

      "customer_disclosure": {

```

```

    "fine_print": [
      "Pay $693.76 today to lock 6 months at $91.01/month all-in.",
      "After 2026-11-15, monthly adjusts to $95.40 all-in (rent +5% adjustment).",
      "Insurance is changeable – see insurance.options for higher coverage tiers."
    ]
  },
  {
    "slot": 2, "label": "Best Alternative",
    "match_flags": { "alternative_strategy": "same_site_2nd" },
    "...": "same shape as slot 1"
  },
  {
    "slot": 3, "label": "Best Price",
    "match_flags": { "relaxed_dims": ["unit_type"], "savings_pct": 18.3 },
    "...": "same shape – strictly cheaper than slot 1"
  }
],

"next_turn": {
  "previous_request_id": "550e8400-e29b-41d4-a716-446655440000",
  "session_id": "sess-conv-7",
  "supported_actions": [
    "more_like_this", "bigger_size", "smaller_size",
    "expand_locations", "different_type", "different_duration"
  ],
  "next_level_allowed": true
},

"reserve_template": {
  "endpoint": "POST /api/reservations/reserve",
  "shared": {
    "session_id": "sess-conv-7",
    "customer_id": "user_xyz_123",
    "previous_request_id": "550e8400-e29b-41d4-a716-446655440000"
  },
  "slots": [
    {
      "site_code": "L017",
      "unit_id": 107197,
      "concession_id": 11872,
      "plan_id": 10,
      "quoted_rate": 149.00
    },
    {
      "site_code": "L017",
      "unit_id": 107298,
      "concession_id": 11873,
      "plan_id": 10,
      "quoted_rate": 159.00
    },
    null
  ]
}
}

```

Slot semantics

Slot Label	What it is	Distinguished by match_flags
1 Best Match	Cheapest unit matching every filter	<code>alternative_strategy: null</code> , <code>relaxed_dims: []</code>
2 Best Alternative	Most convenient alternative	<code>alternative_strategy: same_site_2nd / neighbour_close / neighbour_far</code>
3 Best Price	Strictly cheaper unit at the same site	<code>relaxed_dims</code> : which dim was dropped; <code>savings_pct</code>

When the strict-filter pool is empty, the engine runs **pool rescue** — it auto-relaxes dimensions until candidates appear. The bot doesn't need to retry; the recommend response surfaces this via:

- `stats.saturation_signal: true`
- `stats.pool_rescue_step` — which dim got relaxed first

- per-slot `match_flags.relaxed_dims` — what each slot's match dropped

Combined with directional actions (`bigger_size` / `smaller_size` open the range fully in that direction), this means an action that lands on an empty bucket still returns the closest available match — never zero slots in normal operation.

The response is guaranteed to contain ≥ 2 **slots in normal operation**. Slot 3 is best-effort; null if no strictly-cheaper unit exists at the site.

Reservation fee (`reservation_fee`)

Each slot carries the amount to charge the customer when they confirm the reservation, in the slot's site local currency:

Field	Type	Meaning
<code>reservation_fee</code>	number	Amount to charge to confirm the booking
<code>reservation_fee_source</code>	"override" "default"	How it was resolved

Resolution rules:

- `override` — site has a configured reservation fee in `mw_reservation_fees` (managed by Revenue at `/tools/reservation-fees`). That value is used as-is.
- `default` — no override row for the site. Falls back to **one month of `std_rate`** for the slot's unit — the historical default for the chatbot flow.

The two-tier model lets Revenue ops set a flat fee per site (e.g. ₩50,000 across all units at L031) while every other site continues with the per-unit standard-rate default — no bot change required.

`mode=quote` — single-unit pricing

For "what does unit X cost?" — pass `mode: "quote"` plus EITHER `filters.unit_id: [N]` (when you have the internal id) OR `filters.unit_name: ["4120"]` (when the customer named the unit by its printed number). Returns one slot keyed to that unit. Pricing is **cent-identical** to what `mode=recommendation` returned for the same unit.

`mode=quote` is **stateless** — it does not consume a recommendation log row, does not enforce `request_id` uniqueness, and does not participate in the multi-turn chain-depth cap (\$5). The bot can re-price freely mid-conversation without affecting the L1/L2/L3 chain.

By `unit_id`:

```
{
  "mode": "quote",
  "duration_months": 6,
  "filters": { "location": ["L017"], "unit_id": [107197] },
  "context": { "channel": "chatbot", "request_id": "...", "session_id": "...", "customer_id": "..." }
}
```

By `unit_name` (customer-friendly):

```
{
  "mode": "quote",
  "duration_months": 6,
  "filters": { "location": ["L017"], "unit_name": ["4120"] },
  "context": { "channel": "chatbot", "request_id": "...", "session_id": "...", "customer_id": "..." }
}
```

5 • Multi-turn search

The recommender is **hierarchical, max 3 levels deep** per booking flow. Pass `previous_request_id` + action to continue from a prior turn.

Level 1	Initial recommend	— no <code>previous_request_id</code>
Level 2	Continuation off L1	— <code>previous_request_id</code> = L1. <code>request_id</code>
Level 3	Continuation off L2	— <code>previous_request_id</code> = L2. <code>request_id</code>
Level 4	REJECTED with HTTP 400 — customer must accept a slot or pivot to a fresh L1	

`stats.recommendation_level` (diagnostic, 1/2/3 today) tells you where you are.
`next_turn.next_level_allowed` flips to false once the configured cap is reached — **that is the only safe signal to stop chaining**. The depth cap is admin-tunable (1-6); **do not hard-code on `recommendation_level == 3`** or your bot will break the moment ops widens or tightens it.

Action vocabulary

All actions require `previous_request_id`.

action	What the engine does
<code>more_like_this</code> (paired with <code>picked_slot=1\ 2\ 3</code>)	Slot-specific tightening. Slots 1/3 → <code>size_range</code> ±1 bucket. Slot 2 → next-nearest site.
<code>bigger_size</code>	Open <code>size_range</code> to every bucket STRICTLY bigger than the current one. Slot 1 + Slot 2 honor the direction . Slot 3 (Best Price) may still relax size to surface a cheaper option (its <code>match_flags.relaxed_dims</code> reports it).
<code>smaller_size</code>	Open <code>size_range</code> to every bucket STRICTLY smaller than the current one. Same Slot-3 caveat.
<code>expand_locations</code>	Add nearest neighbour sites (within <code>max_distance_km</code>) to <code>filters.location</code>
<code>different_type</code>	Drop the <code>unit_type</code> filter
<code>different_duration</code>	Analytics signal — bot also changes <code>duration_months</code> so the engine re-quotes against the new length
<code>cheaper_only</code>	Keeps every filter intact and surfaces the cheapest available units in the same pool. Pair with <code>previous_request_id</code> so already-shown <code>unit_ids</code> are auto-excluded — slot 1 becomes the next-cheapest match the customer hasn't seen yet. Use when the customer says "anything cheaper but same size/type/etc.?"

Decision tree

Customer says...	Bot's next call
"I'll take slot 1 / slot 2"	Skip recommend → POST /reserve
"More like that one" / "Anything similar to slot 2?"	recommend with <code>previous_request_id</code> + <code>picked_slot</code> + action: " <code>more_like_this</code> "
"Other options?" / "What else?"	recommend with <code>previous_request_id</code> alone — auto-excludes shown units
"Something bigger" / "Something smaller"	recommend with <code>previous_request_id</code> + action: " <code>bigger_size</code> " (or <code>smaller_size</code>)
"What about nearby locations?"	recommend with <code>previous_request_id</code> + action: " <code>expand_locations</code> "
"Different unit type?"	recommend with <code>previous_request_id</code> + action: " <code>different_type</code> "
"What if I lease for 12 months?"	recommend with new <code>duration_months</code> + action: " <code>different_duration</code> "
"Anything cheaper at the same size/type?"	recommend with <code>previous_request_id</code> + action: " <code>cheaper_only</code> ". No filter mutation; the auto-exclusion of shown units surfaces the next-cheapest match.
"Actually I want something completely different"	Fresh L1: new filters, new <code>request_id</code> , keep <code>session_id</code> + <code>customer_id</code> , drop <code>previous_request_id</code> / <code>picked_slot</code> /action

6 • POST /api/reservations/reserve

Holds the unit and creates a tenant + reservation record. **No payment is taken.**

Minimal request

```
curl -X POST https://backend.extraspace.com.sg/api/reservations/reserve \
-H "X-API-Key: $ESA_API_KEY" \
-H "Content-Type: application/json" \
-d '{
  "site_code": "L017", "unit_id": 107197, "concession_id": 11872,
  "first_name": "Jane", "last_name": "Tan", "phone": "+6591234567",
  "needed_date": "2026-05-15",
  "session_id": "sess-conv-7",
  "customer_id": "user_xyz_123"
}'
```

Bridge from reserve_template

The recommend response includes a `reserve_template` object. Bot integration is a single spread regardless of which slot the customer picked:

```
const rt = recommendResponse.reserve_template;
const pickedIndex = 1; // 0-based: slot 1 = index 0, slot 2 = index 1, slot 3 = index 2

const body = {
  ...rt.shared,
  ...rt.slots[pickedIndex],
  first_name, last_name, phone, email, needed_date, expires_date,
};

fetch('/api/reservations/reserve', { method: 'POST', body: JSON.stringify(body) });
```

`rt.slots[N]` is null when slot N+1 was empty in the recommend response — guard before spreading. `concession_id` of 0 means standard rate; pass 0, never null.

Body fields

Field	Required Notes	
site_code	yes	from the picked slot
unit_id	yes	from the picked slot
concession_id	yes	from the picked slot. 0 = standard rate — pass 0, never null
plan_id	optional	from the picked slot; attribution signal
first_name, last_name, phone	yes	tenant identity
email, mobile, address, city, postal_code, country	optional	
needed_date	optional	YYYY-MM-DD, default = tomorrow
comment, quoted_rate, source_name	optional	
session_id, customer_id	yes	must match the recommend call
previous_request_id	optional	the picked slot's recommend request_id — strongest attribution

Response

```
{
  "success": true,
  "site_code": "L017",
  "unit_id": 107197,
  "tenant_id": "1109544",
  "waiting_id": "848757",
  "global_waiting_num": "809363726",
  "message": "Reservation created"
}
```

Hold on to `waiting_id` and `tenant_id` — both identify the held reservation for any subsequent read / modify / cancel call.

7 • GET /api/reservations/move-in/cost

Authoritative price for the held reservation. Call this **right before showing the customer the total**.

Minimal request

```
curl -G "https://backend.extraspace.com.sg/api/reservations/move-in/cost" \
-H "X-API-Key: $ESA_API_KEY" \
--data-urlencode "site_code=L017" \
--data-urlencode "unit_id=107197" \
--data-urlencode "concession_id=11872" \
--data-urlencode "insurance_id=1" \
--data-urlencode "move_in_date=2026-05-15" \
--data-urlencode "waiting_id=848757" \
--data-urlencode "variant=reservation"
```

Query parameters

Param	Required	Notes
site_code	yes	
unit_id	yes	
concession_id	optional	same value used on /reserve; default 0
insurance_id	optional	the slot's selected insurance, or 0 for none
move_in_date	optional	default = tomorrow
variant	optional	standard (default) reservation 28day push_rate. Use reservation for a held reservation.
waiting_id	required when variant=reservation	from /reserve
promo_id	optional	only with variant=reservation

Response

```
{
  "site_code": "L017", "unit_id": 107197, "move_in_date": "2026-05-15",
  "tenant_rate": 80.50, "discount": 24.15,
  "total": 238.71,
  "charges": [
    { "description": "First Monthly Rent Fee", "amount": 80.50, "tax": 7.25, "total": 87.75 },
    { "description": "Administrative Fee", "amount": 30.00, "tax": 2.70, "total": 32.70 },
    { "description": "Security Deposit", "amount": 296.00, "tax": 0.00, "total": 296.00 },
    { "description": "First Month Insurance", "amount": 3.00, "tax": 0.27, "total": 3.27 }
  ]
}
```

The `total` field is the authoritative amount.

8 • Reservation lifecycle

For reading or modifying a held reservation.

Read

```
curl "https://backend.extraspace.com.sg/api/reservations/848757?site_code=L017" \
-H "X-API-Key: $ESA_API_KEY"
```

Returns the reservation's current state.

Modify

```
curl -X PUT https://backend.extraspace.com.sg/api/reservations/848757 \
-H "X-API-Key: $ESA_API_KEY" \
-H "Content-Type: application/json" \
-d '{
  "site_code": "L017",
  "needed_date": "2026-05-20",
  "first_name": "Jane",
  "phone": "+6591234567",
  "comment": "Customer changed move-in date"
}'
```

Only fields you include in the body are written. Returns `{"success": true, "waiting_id": ..., "message": "Reservation updated"}`.

Cancel

```
curl -X PUT https://backend.extraspace.com.sg/api/reservations/848757/cancel \
-H "X-API-Key: $ESA_API_KEY" \
-H "Content-Type: application/json" \
-d '{"site_code": "L017"}'
```

Idempotent — cancelling an already-cancelled reservation returns success.

9 • End-to-end

A 3-turn chat ending in a held reservation with confirmed price.

```

// — Turn 1 – fresh recommend —————
POST /api/recommendations
{
  "duration_months": 6,
  "filters": { "location": ["L017"], "size_range": ["30-35"] },
  "context": {
    "channel": "chatbot",
    "request_id": "req-aaaa-1111",
    "session_id": "sess-conv-7",
    "customer_id": "user_xyz_123"
  }
}
// → 3 slots · stats.recommendation_level = 1 · next_level_allowed = true

// — Turn 2 – "show me something bigger" —————
POST /api/recommendations
{
  "duration_months": 6,
  "filters": { "location": ["L017"], "size_range": ["30-35"] },
  "context": {
    "channel": "chatbot",
    "request_id": "req-bbbb-2222",
    "session_id": "sess-conv-7", // unchanged
    "customer_id": "user_xyz_123", // unchanged
    "previous_request_id": "req-aaaa-1111",
    "action": "bigger_size"
  }
}
// → 3 fresh slots in 35-40 range · stats.recommendation_level = 2

// — Customer picks slot 1 – hold the unit —————
// rt = response.reserve_template; pickedIndex = 0 (slot 1)
POST /api/reservations/reserve
{
  // spread: { ...reserve_template.shared, ...reserve_template.slots[0], <customer fields> }
  "session_id": "sess-conv-7",
  "customer_id": "user_xyz_123",
  "previous_request_id": "req-bbbb-2222",
  "site_code": "L017",
  "unit_id": 107298,
  "concession_id": 11872,
  "plan_id": 10,
  "first_name": "Jane", "last_name": "Tan", "phone": "+6591234567",
  "email": "jane@example.com",
  "needed_date": "2026-05-15"
}
// → tenant_id = 1109544, waiting_id = 848757

// — Confirm authoritative price right before showing the total —————
GET /api/reservations/move-in/cost
  ?site_code=L017
  &unit_id=107298
  &concession_id=11872
  &insurance_id=1
  &move_in_date=2026-05-15
  &waiting_id=848757
  &variant=reservation
// → { total: 238.71, charges: [...] }

```

10 • Reference

Unit type codes

Code Meaning

W	Walk-in (full-size storage room)
WN	Wine storage
L	Locker (smaller, rack-style)
U	Upper-position locker
M	Mid-position locker
LL	Lower-position locker

Climate codes

Code Meaning

A	Air-conditioned
AD	Air-conditioned + dehumidified
NC	Non-climate
RF	Refrigerated (specialised)

Plan vs. concession — what to display

Each slot exposes both:

Field	Audience	Example	Rule
plan_name	Customer-facing	"Moving Season SG"	Display this to the end customer. It's the operator-curated brand.
concession_name	Internal	"SS-TAC-Move-R-30%"	This is the SiteLink operator code. Do not show to customers.
discount_summary	Customer-facing	"30% off every month"	Pre-rendered human-readable summary; safe to display verbatim.

Pass `concession_id` (the integer) verbatim through `/reserve`. Use `plan_name` + `discount_summary` for any UI copy the customer sees.

Channel codes

Code Meaning

chatbot	Conversational bot integration
web	Direct web booking
api	Generic API caller
admin	Admin tooling

Common error responses

HTTP	Where	What it means
400	any endpoint	Invalid input — body has <code>error</code> field with the specific failure
401	any endpoint	Missing / wrong X-API-Key
403	any endpoint	API key doesn't have the required scope
400	/recommendations	recommendation chain exceeds max depth (3) — drop previous_request_id and start a fresh L1
200 + success: false	/reserve	Unit got rented or unavailable. Body's <code>message</code> explains. Re-recommend with <code>constraints.exclude_unit_ids: [<that unit_id>]</code> .
429	any endpoint	Rate limit exceeded; <code>retry_after</code> seconds in body

11 • Best practices

1. **Send all four context IDs on every recommend call.** Keep `session_id` + `customer_id` consistent across `recommend` → `reserve`.
 2. **Mint a fresh request_id per turn.** Don't reuse it; it's the link key for `previous_request_id` on the next turn.
 3. **Re-fetch /move-in/cost immediately before showing the total** to the customer. Pricing can shift if a session is long-running.
 4. **Treat HTTP 200 + success: false as a failure** (most often "unit no longer available"). Re-recommend with the failed `unit_id` in `constraints.exclude_unit_ids`.
 5. **Stop chaining at level 3.** Watch `next_turn.next_level_allowed` — `false` means encourage the customer to pick a slot or rephrase.
 6. **concession_id=0 = standard rate.** Pass `0`, never `null`, when the picked slot has no discount.
 7. **Don't share API keys.** One key per integration; rotate via Extra Space Asia ops if compromised.
-

12 • Support

- **Integration questions** — contact your Extra Space Asia integration manager.
 - **Liveness check** — `GET /api/health` returns `{"status": "ok"}` when the API is up.
 - **API key management** — `https://backend.extraspace.com.sg/api-keys/` (auth required).
-

13 • Changelog

Versioning follows [semver](#): MAJOR.MINOR.PATCH. **MINOR** bumps add optional response fields or new endpoints and are backwards-compatible — existing integrations keep working without changes.

1.2.0 — 2026-05-12

Changed

- `reserve_template` restructured to a nested shape with `shared` + per-slot `slots[]` array (indexed 0-2 to mirror `slots[]`). Each slot's block now includes `quoted_rate` (= `std_rate`). Bot integration is now a single merge regardless of which slot the customer picked. **Note:** this is a breaking change to `reserve_template` only — all other response fields are unchanged. No partner has shipped against the old shape, so no compat shim is provided.

1.1.0 — 2026-05-12

Added

- Each slot in `POST /api/recommendations` responses now carries `reservation_fee` and `reservation_fee_source` ("override" | "default"). See [§4 • Reservation fee](#). Use this as the amount to charge to confirm the booking — it's authoritative over `std_rate` when Revenue has set a per-site override.

1.0.0 — 2026-04-29

- Initial public release: `/api/recommendations` (recommendation + quote modes), `/api/reservations/reserve`, `/api/reservations/move-in/cost`, reservation lifecycle endpoints.